

Trie Data Structure

Datová Struktura Trie

Tuan Phong Huynh

Bachelor Thesis

Supervisor: prof. Ing. Michal Krátký, Ph.D.

Ostrava, 2025

Bachelor Thesis Assignment

Student:

Tuan Phong Huynh

Study Programme:

B0613A140010 Computer Science

Title:

Trie Data Structure

Datová struktura trie

The thesis language:

English

Description:

A Trie is a memory data structure with a compact representation, enabling point queries on strings of variable or fixed lengths. Efficient implementation of a Trie is not always straightforward, especially for larger alphabets of string symbols.

1. Study different variants of the Trie data structure.
2. Implement the chosen variant, focusing on larger alphabets of string symbols.
3. Test your own implementation and compare it with the performance of other data structures.

References:

[1] Aoe, J.-I., Morimoto, K. and Sato, T. (1992), An efficient implementation of trie structures. Softw: Pract. Exper., 22: 695-721. <https://doi.org/10.1002/spe.4380220902>

Extent and terms of a thesis are specified in directions for its elaboration that are opened to the public on the web sites of the faculty.

Supervisor:

prof. Ing. Michal Krátký, Ph.D.

Date of issue: 01.09.2024

Date of submission: 30.04.2025

Study programme guarantor: doc. Mgr. Miloš Kudělka, Ph.D.

In IS EDISON assigned: 25.10.2024 11:09:56

Abstrakt

Tato práce se zaměřuje na datovou strukturu Trie, typ datové struktury, která zpracovává hlavně prefixy řetězců. Struktura Trie je velmi oblíbená pro aplikace, jako je vytváření slovníků, kontrola pravopisu a automatické doplňování. Výkon však může být náročný, pokud Trie pracuje s velkými řetězci nebo velkou abecedou, což vede k různým problémům s jejími možnostmi načítání a ukládání. Hlavní výzvou je výběr vhodné datové struktury pro zpracování znaků v uzlech Trie na základě faktorů, jako je velikost abecedy, kterou dokáže zpracovat, výkon operací a složitost implementace. Proto se v této práci budeme zabývat variantami datové struktury Trie a budeme zvažovat jejich struktury a operace, abychom usnadnili výběr optimálního přístupu pro zpracování velkých řetězců a abeced. Nakonec porovnáme výkon této varianty se sekvencí maticovou strukturou, abychom posoudili její schopnosti zpracování řetězců a zdůraznili její potenciál pro praktické aplikace.

Klíčová slova

Datová struktura Trie; prefixy řetězců; variace datové struktury Trie; porovnání výkonu; praktické aplikace

Abstract

This thesis focuses on the Trie data structure, a type of data structure that mainly handles the prefixes of the strings. Tries are very popular for applications such as building dictionaries, spell-checking, and auto-completion. However, performance can be challenging if the Trie works with large strings or a large alphabet, leading to various issues in its retrieval and storage capabilities. The main challenge is to select an appropriate data structure for handling the characters in the nodes of a Trie, based on factors such as the size of the alphabet that it can handle, operations performance, and implementation complexity. Therefore, in this thesis, we will study variations of the Trie data structure, considering their structures and operations to facilitate the selection of the optimal approach for processing large strings and alphabets. Finally, we will compare the performance of this variation with sequential array structure to assess its string-processing capabilities, highlighting its potential for practical applications.

Keywords

Trie data structure; prefixes of the strings; variations of the Trie data structure; compare the performance; practical applications

Acknowledgement

I would like to thank and express my gratitude to my supervisor, prof. Ing. Michal Krátký, Ph.D., for his support, valuable guidance and patience during my studies and development of this thesis.

Contents

List of symbols and abbreviations	6
List of Figures	7
List of Tables	8
1 Introduction	9
2 Trie Data Structures	11
2.1 Overview of Trie Data Structure	11
2.2 Node Structures	12
2.3 Operations implementations	13
2.4 Node implementations	23
2.5 Implementation Description	26
3 Experimental Setup and Details	29
3.1 Overview	29
3.2 Experimental Setup	29
3.3 Experimental Details	32
4 Conclusion	43
Bibliography	44
Appendices	45
A Command-Line Interface	46

List of symbols and abbreviations

std	– Standard Library
UTF	– Unicode Transformation Format
txt	– Text File
UML	– Unified Modeling Language
ASCII	– American Standard Code for Information Interchange

List of Figures

2.1	An example of a standard trie	12
2.2	Example of linking between nodes in fixed-size array	13
2.3	Example of sorted array	17
2.4	Example of Trie with the hash-table node	17
2.5	UML object diagram of non-sorted array trie	25
2.6	UML object diagram of hash table trie	25
2.7	UML class diagram of Non-Sorted Array Trie	26
2.8	UML class diagram of Sorted Array Trie	27
2.9	UML class diagram of Hash Table Trie	28
3.1	Histogram of $ C $ in English dataset	32
3.2	Histogram of $ C $ in Vietnamese dataset	33
3.3	Histogram of $ C $ in Czech dataset	33
3.4	Histogram of $ C $ in Chinese dataset	34
3.5	Histogram of $ C $ in Mixed dataset	34
3.6	Insert time (in seconds) across datasets	36
3.7	Insert memory usage (in MBs) across datasets	37
3.8	Insert memory (in MBs) across datasets	41

List of Tables

3.1	Machine and software configuration used in the experiment	30
3.2	Summary of datasets used in the experiment	30
3.3	Word length statistics for each dataset	31
3.4	Insert time (in seconds) for different trie types and sequential array across datasets .	35
3.5	Insert memory (in MBs) for different trie types and sequential array across datasets	35
3.6	Search time (in seconds) for different trie types and sequential array across datasets	38
3.7	Prefix search time (in seconds) across datasets and data structures.	38
3.8	Deletion time (in seconds) for different trie types and sequential array across datasets	39
3.9	Insert memory (in MBs) for different trie types across datasets	40
3.10	Prefix search time (in seconds) comparing linear and quadratic probing.	42

Chapter 1

Introduction

The trie is also known as a digital tree or prefix tree [1]. In real life, the foundation for some things we are highly familiar with is the autocomplete system that we can see in the search engines like Google or Bing [2]. This is a helpful data structure for storing and searching strings. Unlike other forms of search trees, it has a huge advantage in operation performance, where each operation's time complexity is proportional to the length of the string instead of the number of keys stored [3, 4]. However, the efficient implementation of trie structures can be difficult when dealing with a large alphabet.

The aim of this thesis is to perform an in-depth examination of the various variants of trie, such as Nonsorted Array, Sorted Array, and Hash Table variants, to deduce an efficient solution when big alphabets are in use. All these Trie variations are suited for a wide range of applications because of their respective distinctive characteristics pertaining to memory usage, computational efficiency, and operational behavior.

After the theoretical analysis, the implementation part will be done in the thesis. Here, the variant of trie that is selected based on the results from the theoretical part will be implemented on datasets with big alphabets. In particular, Vietnamese text is used as a dataset, this dataset has a rich set of diacritics and compound characters, these strings are encoded using UTF-8 to ensure it can test in a realistic condition for trie under large-alphabet scenarios. Its performance shall then be evaluated and compared to other widely used data structures. The comparison is done to bring out the strengths and weaknesses of the chosen variant of trie, so one could get a comprehensive understanding of how effective it would be in handling large alphabets.

The structure of this thesis is organized:

- Chapter 2: Explains the structures and how three variants of the node implementation the non-sorted array, sorted array and hash table trie work.
- Chapter 3: Describes the experimental setup. It will provide all the datasets used (including English and Vietnamese text encoded in UTF-8) and the testing environment. Presents the

experimental results and a preliminary evaluation of proposed solutions to address the problems identified in earlier trie implementations, it will show all the performance measurements of insertion, search and deletion operations, and provides an analysis of all tries in each dataset in used.

- Chapter 4: conclusion the thesis by summarizing and highlighting the findings.

Chapter 2

Trie Data Structures

2.1 Overview of Trie Data Structure

Trie can be pronounced as "tree" or "try" [3]. The term is taken from the word "retrieval" [1]. In computer science, a trie is also known as a digital tree or prefix tree and is a type of search tree [3, 5]. The trie was designed to optimize storage and searching for sets of string in case prefixes matching [5]. Unlike other binary search trees or balanced trees, tries represent for characters not entire strings, where each node stores a single character. In real life tries is used in applications requiring prefix-based searches, such as autocomplete, spell-checking, and IP routing [3, 6].

Importantly, tries do not store complete strings at any single node. Instead, a string is represented as a sequence of nodes, each character from the root to the last node (or an end-of-word marker) [7]. This characteristic allows the trie to share common prefixes between different words, reducing memory usage when multiple strings share the same prefixes.

Figure 2.1 gives an example of a standard trie, which shows that the first node is always null, representing the root of the trie. Following this node, it expands to other branches; as seen in Figure 2.1, words such as "cat," "can," "bed," and "by" are stored in each node when it finishes the word, with shared prefixes being merged into common paths. For example, "cat" and "can" share the prefix "ca," and the trie branches only at the point where the words differ, saving memory usage. It is important to know that complete strings are not stored in the Trie; instead, they are represented by the sequence of nodes along a path from the root to an end-of-word marker.

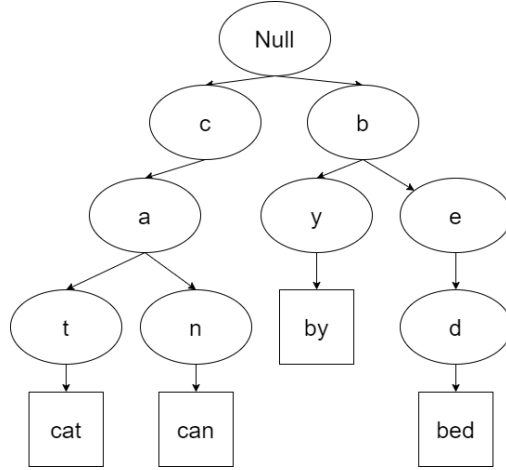


Figure 2.1: An example of a standard trie

In the modern trie implementations, it is necessary to support international character sets, those characters are represented as Unicode code points [8], instead of a byte or ASCII character, this is important when using UTF-8 encoding, where characters like "á" or "đ" contains multiple bytes but when processing in tries, those Unicode characters will be treated as code points.

2.2 Node Structures

Before describing the basic operations, it is necessary to describe the structure of a node, since it determines the algorithm of the operations. Because trie is a types of search trees, it also has nodes, except the root node, a node points to child nodes is the parent node of the child nodes [3, 9], each node will point to child nodes or a null node [10]. A node can have multiple pointers, let $|S|$ be the size of the alphabet, a node can include maximally $|S|$ characters. Let $|C|$ be the actual number children in a node ($|C| \leq |S|$), this is important when analyzing time complexity, the algorithm can depend on $|C|$ in dynamic structures like sorted array or hash tables.

To handle characters in nodes, the following data structures are commonly used in a node:

- Fixed-size array: provides constant-time access, but only for small alphabets like ASCII characters. For large alphabets like Unicode, $|C| = |S|$ must be assumed which will waste the memory for unused slots. All characters are not linked at the beginning in a trie structure, see Figure 2.2. Each node of a trie will contain only characters that are necessary to store to complete the strings [3]. In addition, characters are stored implicitly, which means the position of each character is noticed via the links between nodes instead of stored directly in each node. In a node, there are only stored essential links to the prefixes of the strings [3].

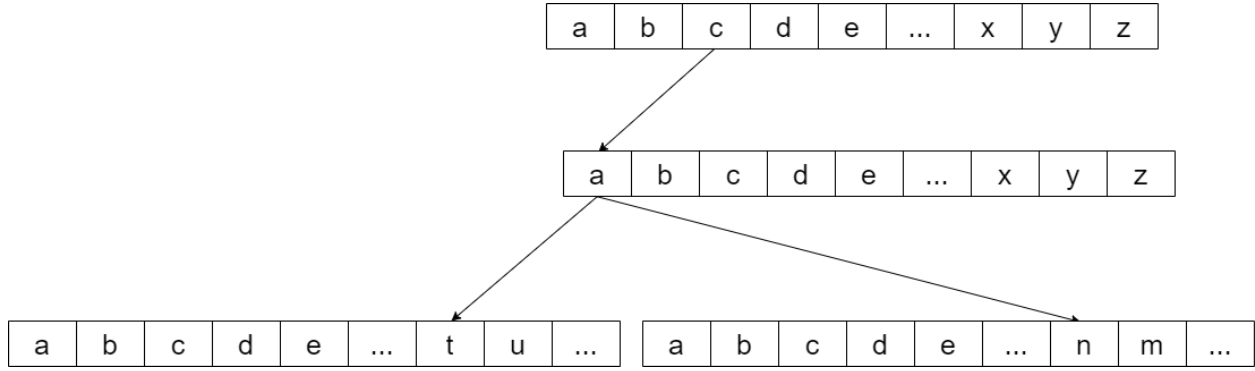


Figure 2.2: Example of linking between nodes in fixed-size array

- **Non-Sorted Array:** Stores character-node pairs in an unsorted array. This structure provides simple insertion with time complexity $O(1)$. However, the time complexity of the query operation is $O(|C|)$.
- **Sorted Array:** Similar to the non-sorted array but keeps character-node pairs in sorted order. This structure requires the insertion shifting the elements to keep children in trie in order, resulting in $O(|C|)$. Meanwhile, searching benefits from binary search with time complexity $O(\log(|C|))$.
- **Hash table:** Uses a hash function to map characters to child nodes. It provides average-case constant-time performance for operations [11] which is $O(1)$. However, in the worst-case scenario, such as when the hash function is bad configured or too many collisions occur, it will lead to $O(|C|)$.

2.3 Operations implementations

In this section, the basic operations of trie will be explained such as: insertion, searching and deletion. For each data structure used in node implementation, the algorithms are the same but the way to access the child node, select position of the child nodes are different.

Also, in the standard ASCII table, there are totally 128 characters, which is different from real life. In real life, there are various languages that have special characters like Vietnamese or Czech. These characters create a challenge for fixed-size array and leads to wasted memory [1, 11, 12, 13]. Instead, uses UTF-8 encoding which supports a larger set of characters (over a million characters) [8]. Therefore, UTF-8 is more realistic for implementing a trie. ASCII is just a special case of UTF-8 which means it can be used when $|S|$ is small. When works with UTF-8 encoded, each character must be treated as a code point which refers to a unique numerical value assigned to each character in the Unicode standard [14]. For example, the character 'á' is a single code point and should be

treated as "U+00E1" code point value during traversal. Using proper UTF-8 decoding in C++ or library like `codecvt` is necessary to iterate correctly through code point.

2.3.1 Insertion

The insertion algorithm iterates over each character of the input string and compares each character with the character in the trie until it finds the right position to add nodes.

1. Non-Sorted Array: This approach uses a dynamic array `children` to store pairs of (character, node) in an unsorted order. It helps to insert characters without maintaining any order of characters in each node. This data structure is suitable with UTF-8, since characters are treated as code points and added to the flexible array without needing index-based mapping, which is typically used in fixed-size arrays. Theoretically, array access is $O(1)$, but because it is an unsorted array with pair (character - node), the trie has to scan all the trie to check if that character exists or not, if not found, it will add the character at the end of the array, thus, even it is a simple data structure, it has to check for duplication, at the result, the time complexity is $O(|C|)$ where $|C|$ is the actual number of children in a node. The insert operation for the non-sorted array is depicted in Listing 2.1.

```
insert(string):
    node = root // starts at root
    for character ch in string // for each character in the input string
    {
        found = false // initialize a flag to track if character is already
                        // in current node's children
        for (c, child) in node.children // traverse the array children
            pairs (character, node)
        {
            if c == ch // if found
            {
                node = child // move to the next child node
                found = true // mark found
                break // exit the loop
            }
        }
        if not found // if not found
        {
            newNode = new TrieNode() // create a new node
            node.children.append((ch, newNode)) // add the character into
            the new node
        }
    }
```

```

        node = newNode // move to the newly created child node
    }
}
node.isEnd = true // after checking all character, mark the current
node is the end of the string.

```

Listing 2.1: Insert of non-sorted array algorithm

2. Sorted Array: This data structure uses a dynamic array of (character, node) pairs, but the array is in sorted order. Each insertion uses binary search to find the existence of the characters and suitable position to insert characters. UTF-8 supports this structure, as each character is compared using operators like $<$ and $==$ in binary search. Unicode characters are handles as long as they are treated as a UTF-8 code points. In theoretical, sorted array would shift elements in the array during insertion, resulting in a worst-case complexity is $O(|C|)$. However, in this implementation, since the binary search is used before the insertion, time complexity is improved with $O(\log|C|)$ and $|C|$ is the actual number children in a node. The insert operation for the sorted array is depicted in Listing 2.2.

```

insert(string):
    node = root // starts at the root
    for character ch in string // for each character of the input string
    {
        index = binarySearch(node.children, ch) // search if characetr
            already exists
        if index != -1 // if character found in sorted array
        {
            node = node.children[index].node // move to the nxt child node
        }
        else // if it not found
        {
            pos = findInsertPosition(node.children, ch) // find position to
                insert (binary search)
            newNode = new TrieNode() // create a new node
            insert (ch, newNode) at pos in node.children // insert into the
                array
            node = newNode // nove to the inserted node
        }
    }
}

```

```
node.isEnd = true // after looping all characters, mark the current
node the end of the word.
```

Listing 2.2: Insert of sorted array algorithm

3. Hash Table: This trie uses hash table for storing child nodes, it applies open addressing and linear probing to handle hash collisions and resize the hash table. Each Unicode character will be treated as a code point and hashed directly to inserted into the hash table with time complexity is $O(1)$. In the worst-case, when table is nearly full or the hash function meets many collisions, it will provide $O(|C|)$ in per node, where $|C|$ is the number of children in a node. The insert operation for the hash table is depicted in Listing 2.3.

```
insert(string):
    node = root // starts at the root
    for each character ch in string // looping each character in the input
        string
    {
        next = node.get(ch) // checking the character in the hash table
        if next is null // if not found
        {
            next = new TrieNode // create a new node
            node.set(ch, next) // add node child into the current node
        }
        node = next // move to the next node
    }
    node.isEnd = true // after looping all characters, mark the current
node the end of the string.
```

Listing 2.3: Insert of algorithm for Hash-table node

For example, see Figure 2.3, inserting "cá" when trie already has "cây", in sorted-array node, 'á' (U+00E1) is less than 'â' (U+00E2), so the order becomes ['á', 'â']. In non-sorted, the insertion order will be contrasted, the children will be ['â', 'á']. Fixed-size is not supported Unicode characters like 'á' or 'â'. In a hash table of capacity 2, both 'á' and 'â' hash to index 1 using Knuth's method. Linear probing places them in separate slots (1 and 0) without ordering, as shown in Figure 2.4.

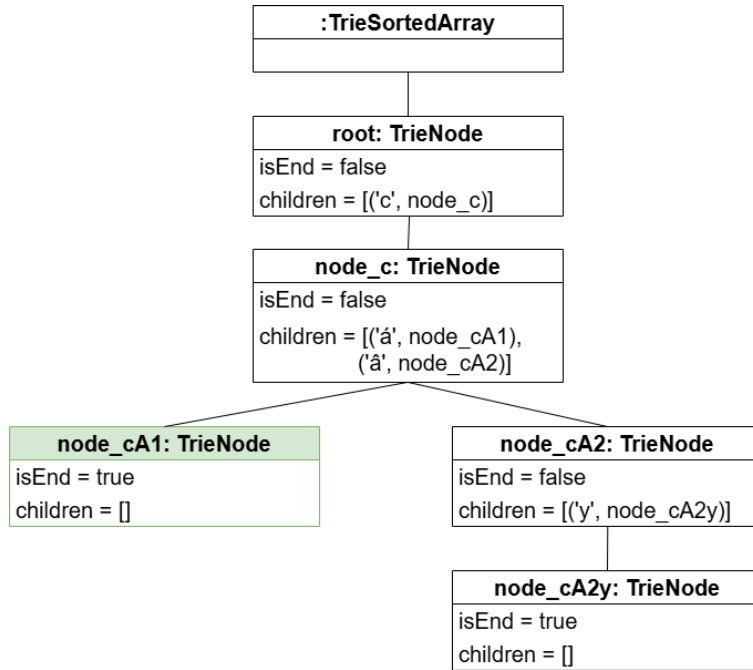


Figure 2.3: Example of sorted array

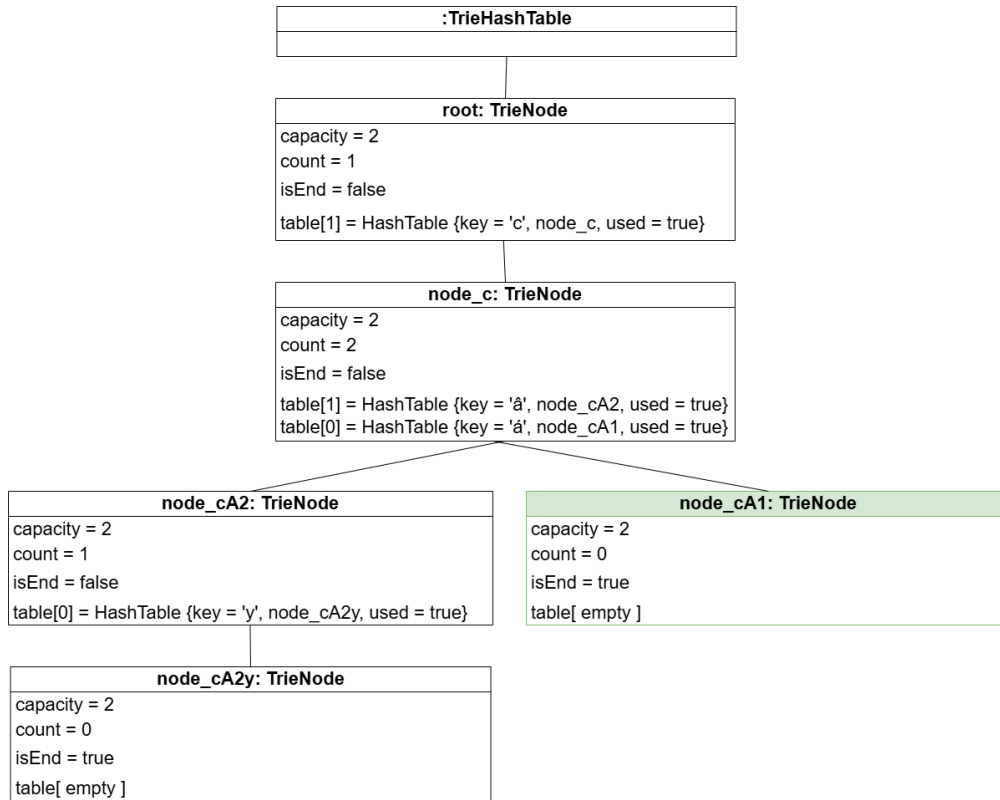


Figure 2.4: Example of Trie with the hash-table node

2.3.2 Searching

Searching algorithm will check if a word exists in the trie by traversing each character in the input. The way access depends on the data structure used to store nodes. For UTF-8, each character is treated as a UTF-8 code point.

1. Non-Sorted Array: This approach is using loop to scan each child in each node and comparing with the input character, if it finds a match it will go to the next child until the input word is done and return isEnd. It is simple and suitable for UTF-8, however, when it has a lot of children in a node, the search requires a long time to return the result. Time complexity is $O(|C|)$ and $|C|$ is the actual number children in a node. The search operation for the non-sorted array is depicted in Listing 2.4

```
search(string):
    node = root // starts at the root
    for character ch in string // for each character in the input string
    {
        found = false // initialize flag to check if the character is found
                           or not
        for (c, child) in node.children // looping the children pair array
        {
            if c == ch // if character matches
            {
                node = child // move to its child node
                found = true // mark found
                break // exit the inner loop
            }
        }
        if not found: // if no matching character
            return false // string is not found
    }
    return node.isEnd // return true if the last node is marked the end of
                        the string.
```

Listing 2.4: Searching algorithm of Trie with non-sorted array node

2. Sorted Array: The Sorted array maintains children in order, using binary search for faster lookups. It is suitable for UTF-8 because the comparisons use code points. Time complexity will be $O(\log|C|)$, where $|C|$ is the actual number children in a node. The search operation for the sorted array is depicted in Listing 2.5.

```

search(string):
    node = root // starts at the root
    for character ch in string // for each character in the input string
    {
        index = binarySearch(node.children, ch) // binary search for
            character in the array
        if index == -1 // if not found
        {
            return false // string does not exist
        }
        node = node.children[index].node // move to the next child node
    }
    return node.isEnd // return true if the last node is marked the end of
        the string.

```

Listing 2.5: Searching algorithm of Trie with sorted array node

3. Hash Table: During search in hash table, each character in the input will be hashed for searching directly in the current node's table. If a character is not found, the word does not exist. Hash table provides the complexity in the average constant with time complexity is $O(1)$. This structure is suitable for UTF-8 code point, each character is hashed for getting the index. The search operation for the hash table is depicted in Listing 2.6.

```

search(string):
    node = root // starts at the root
    for each character ch in string // looping each character in the input
        string
    {
        node = node.get(ch) // checking the character in the hash table
        if node == null // if not found any node
        {
            return false // the string is not found
        }
    }
    return node.isEnd // return true if the last node is marked the end of
        the string.

```

Listing 2.6: Searching algorithm of Trie with hash-table node

Back to Figure 2.3 and Figure 2.4, to search a word "cá", the sorted array found 'c' at the root node and move to its child, uses binary search to find 'á', since 'á' exists as a child and the node is marked `isEnd = true`, the word is found. In contrast, after finding 'c', it will loop linearly to find 'á', the result is slower. The fixed-size does not support UTF-8 character like 'á', hence, the search will fail. Meanwhile, the hash table (with capacity is 2 in each table) hashes each character, after navigating the word 'c' at hash index 1, it will move to the next child node, 'á' is hashed at index 0, and go to its node since the last node is the end of the word "cá", the word is found.

2.3.3 Deletion

The deletion algorithm in Trie is a recursive function, it will traverse the trie and remove a word by unmarking the flag `isEnd`. If any redundant node appears (nodes do not have children and `isEnd = false`), it will remove this node immediately to keep the trie clean.

1. Non-Sorted Array: This deletion algorithm will do a linear scan to navigate the child node, if the child node becomes redundant, it will remove the (code point - node) pair from the array. The structure fully supports for UTF-8 characters, because the array dynamically treats each character as a code point. Time complexity for linear scanning is $O(|C|)$ and $|C|$ is the actual number children in a node. The delete operation for the non-sorted array is depicted in Listing 2.7.

```

delete(node, string, depth): // string will start at root with depth is 0
    if node == null // if node does not exist
    {
        return false // nothing to delete
    }
    if depth == length of string // if reaches the end of the string
    {
        if not node.isEnd // and the node is not mark end
        {
            return false // then the string does not exist
        }
        node.isEnd = false // if it exists, unmark this node
        return node has no children // return true if node has no children
    }
    character = string[depth] // get the current character to compare
    for i in size of node.children - 1 // loop from start to the end of
        the array children
    {
        ch, child = node.children[i] // get the character and the children

```

```

if ch == character{ // compares the the input character with the
    character at the current node
    if delete(child, string, depth + 1) // recursive from the child
        , when it does not has children, that node becomes redundant
    {
        delete node.children[i].second // delete the redundant node
        remove node.children[i] from node.children // remove the
            character of the redundant node from parent
        return node.children is empty and not node.isEnd // delete
            current if it has no children and not end, back to
            delete the parent
    }
    return false // else, end of deletion process
}
}
return false // If character is not found

```

Listing 2.7: Deletion algorithm of Trie with non-sorted array node

2. Sorted Array: In sorted array, the algorithm will use binary search to lookup the node for deletion, for each code point, it will compare the code point with the character in the middle position based on sorted order in the array to find the correct position of the delete character to be deleted. For time complexity will be $O(\log|C|)$ and $|C|$ is the actual number children in a node. The delete operation for the sorted array is depicted in Listing 2.8.

```

delete(node, string, depth): // string will start at root with depth is 0
if node == null // if node does not exist
{
    return false // nothing to delete
}
if depth == length of string // if reaches the end of the string
{
    if not node.isEnd // and the node is not mark end
    {
        return false // then the string does not exist
    }
    node.isEnd = false // if it exists, unmark this node
    return node has no children // return true if node has no children
}

```

```

character = string[depth] // get the current character of the input
index = binarySearch(node.children, character) // uses binary search
        for searching existence of that character in the trie
if index == -1 // if it does not exist
{
    return false // not found
}
child = node.children[index].node // get the child node
if delete(child, string, depth + 1) // recursive from the child, when
    it does not has children, that node becomes redundant
{
    delete child // delete the redundant node
    remove node.children[index] // remove the redundant character from
        the array
    return node.children is empty and not node.isEnd // check if
        current node can be removed
}
return false // return false if no deletion happened

```

Listing 2.8: deletion algorithm of Trie with sorted array node

3. Hash Table: For hash table, same with other data structure, it will loop each character to check it in the hash table, if found, it will traversal to its child node and any redundant nodes are deleted during backtracking state. The time complexity is $O(1)$. The delete operation for the hash table is depicted in Listing 2.9.

```

delete(node, string, depth) // string will start at root with depth is 0
if node is null // if node does not exist
{
    return false // nothing to delete
}
if depth == length of string // if reaches the end of the string
{
    if node.isEnd == false // and the node is mark end
    {
        return false // then the string does not exist
    }
    node.isEnd = false // if it exists, unmark this node
    return node has no children // return true if node has no children
}

```

```

}
character = string[depth] // get the current character of the input
child = node.get(character) // search the character in the hash table
if child == null // if it does not exist
{
    return false // not found
}
if delete(child, string, depth + 1) // recursive from the child, when
    it does not has children, that node becomes redundant
{
    node.remove(character) // remove the child node from the hash table
    return node has no children and not node.isEnd // check if current
        node can be removed
}
return false // return false if no deletion happened

```

Listing 2.9: DeletionInsert of algorithm for Hash-table node

Considering delete the word "cá" in Figure 2.3 and Figure 2.4:

- In sorted array, the algorithm starts at the root and use binary search for finding 'c'. Once found, it moves to its child node (node_c), another binary search is used in the sorted array to find 'á', which has node_cA1, because at this node, it is marked as true, hence, the algorithm will unmark it, and delete this node. During the backtracking state, it checks node_c for removing 'á' from the array and its children. However, node_c still has another child, so the deletion will stop and no more processing.
- In hash table, the same algorithm is applied, the hash table will hash character 'c' to find its position in the first table to find node_c, then 'á' is hashed to find node_cA1, since the node_cA1 is marked as end, it will be unmarked to back to back tracking state to decrease count number of children and remove redundant nodes same as sorted array.
- For non-sorted array the algorithm is the same. However, each step uses linear search to find correct child nodes, making it slower than other data structures.

2.4 Node implementations

Each node in a trie contains two main attributes:

- isEnd: A flag with type boolean to notice the current node is the end of a word.

- children: A dynamic array (vector of pairs) that holds the links to child nodes, depending what kind of trie, it will implement in one of these ways:
 - Non-Sorted Array (vector of pairs): each pair contains a character (code point) and a pointer to point to child nodes. This array is not sorted.
 - Sorted Array (vector of pairs): Each pair contains a character (code point) and a pointer to point to child nodes. Unlike the non-sorted version, this array is sorted each character by using binary search, it also supports insertion, searching and deletion operations.

To contrast, hash table trie has a different structure, it has two main attributes and two other attributes to help resize and hashing the index of the trie:

- isEnd: A flag with boolean type, use for marking the end of a word.
- table: which is the hash table contains keys (characters), nodes to point to children and used, a flag to avoid hash collisions.
- capacity: an index represents a size of a table, it also uses for calculating the hashing value to determine the index of character key.
- count: an index represents the number of children in the current node, it uses for resizing the hash table when the number of children in a table greater than the capacity.

To determine then index in the hash table, each child character has to use a multiplicative hash function based on Knuth's method [15, 4]:

$$index = ((code_point \times 2654435761) \gg 5) \bmod capacity \quad (2.1)$$

To better understanding the structure, see Figure 2.5, with input strings in order are "0,3", "0,1" comparing with hash table structure, see Figure 2.6.

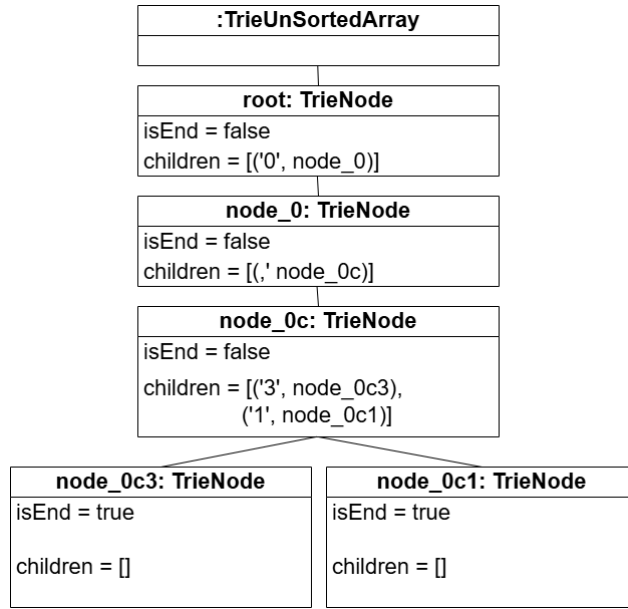


Figure 2.5: UML object diagram of non-sorted array trie

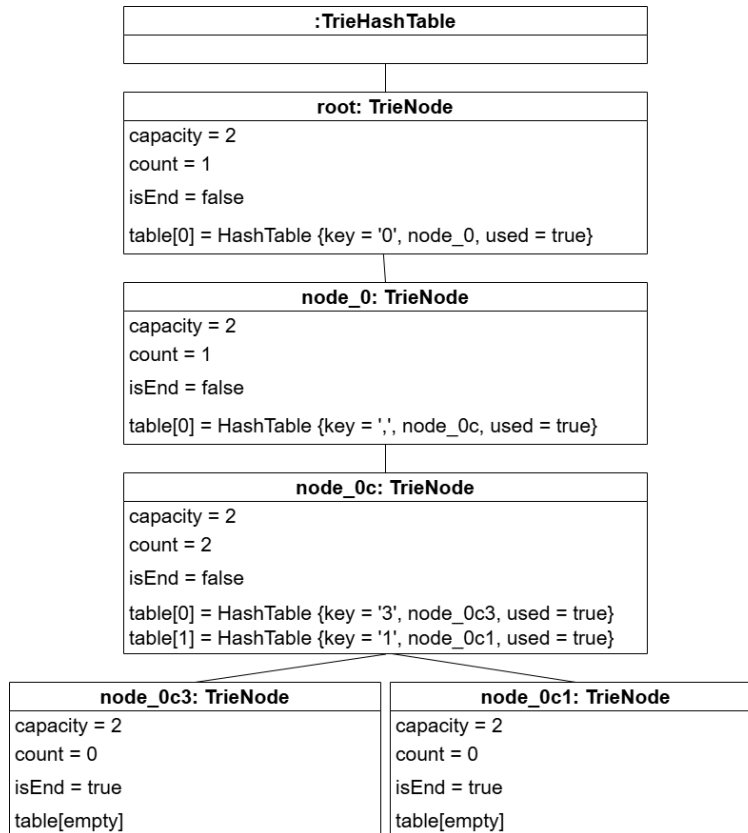


Figure 2.6: UML object diagram of hash table trie

2.5 Implementation Description

This section provides a description of the UML class diagrams to three different trie implementations in this thesis. Each implementation has a **TrieNode** class that handles structures of a trie, and a **Trie** class that implements basic operations such as insertion, searching and deletion.

1. Non-Sorted Array: Figure 2.7 presents the UML diagram of the Non-Sorted Array Trie. The **TrieNode** class stores a dynamic array (vector) of (character, child pointer) pairs without ordering characters. The **TrieUnsortedArray** class contains a root node and provides methods for insertion, searching, deletion, and printing the structure. When inserting or searching for a character, the vector is traversed linearly to find the matching child.

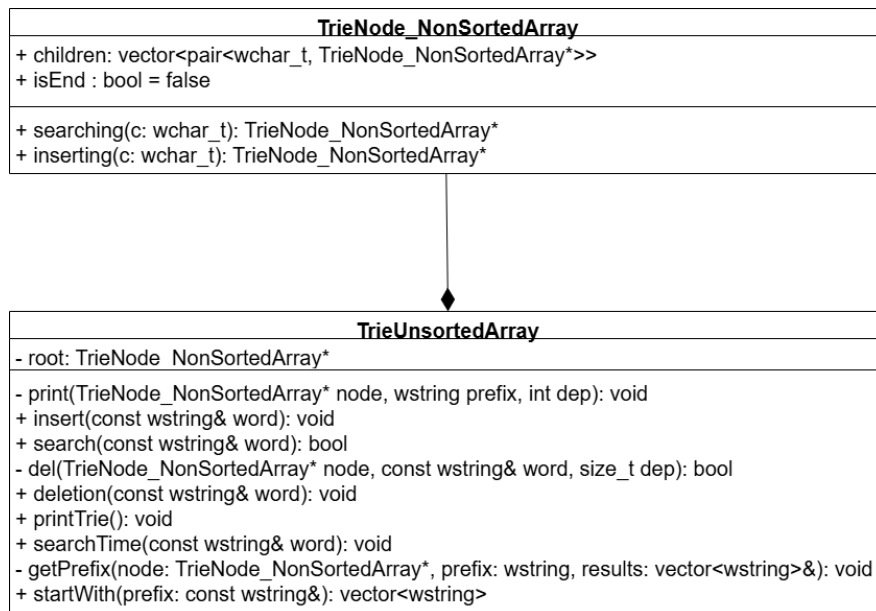


Figure 2.7: UML class diagram of Non-Sorted Array Trie

2. Sorted Array: Figure 2.8 shows the UML diagram Trie with the sorted array node. Similar to the unsorted version, the `TrieNode2` class stores a vector of (character, child pointer) pairs; however, the characters are sorted. The `TrieSortedArray` class contains the root node and provides methods for insertion, searching, deletion, and printing. Binary search methods (`checkChild` and `findPos`) are used to locate children quickly, resulting in improved search and insertion performance compared to the unsorted array.

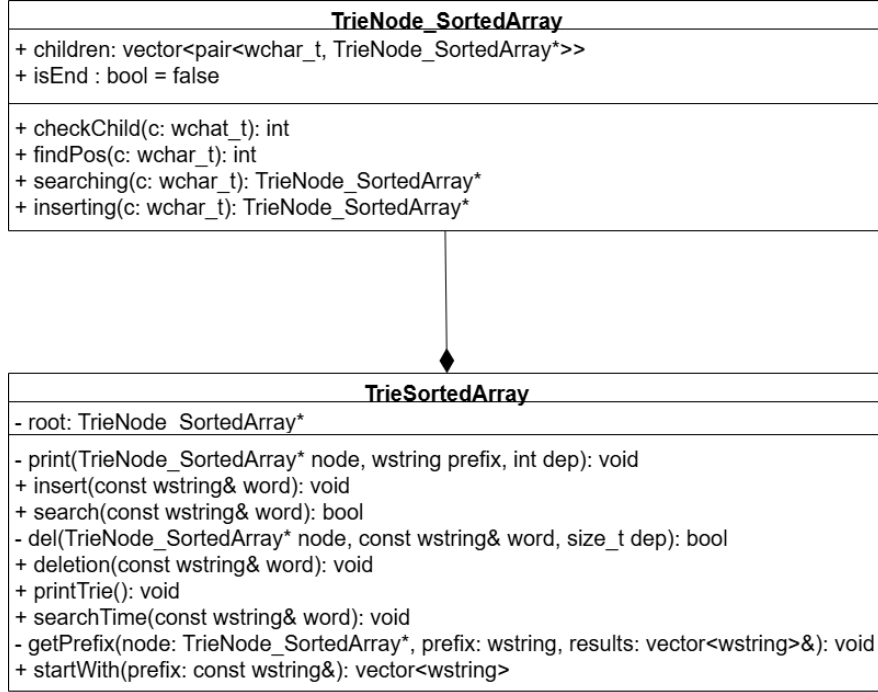


Figure 2.8: UML class diagram of Sorted Array Trie

3. Hash Table: Figure 2.9 depicts the UML diagram of the Hash Table Trie. Each `TrieNode3` consists of a dynamically managed open-addressed hash table of `HashTable` class, where each `HashTable` holds a character key, a child pointer, and a usage flag. The `TrieHashTable` class includes a root node and supports insertion, searching, deletion, and printing operations. Hashing and linear probing are used for optimal search and insertion within each node, making this implementation faster for large alphabets. Linear Probing is a collision resolution strategy. When a collision occurs on insert, we probe the hash table, in a linear, stepwise fashion, to find the next available space in which to store our new object [16], it can be proposed based on the equation:

$$index = (index + 1) \bmod capacity \quad (2.2)$$

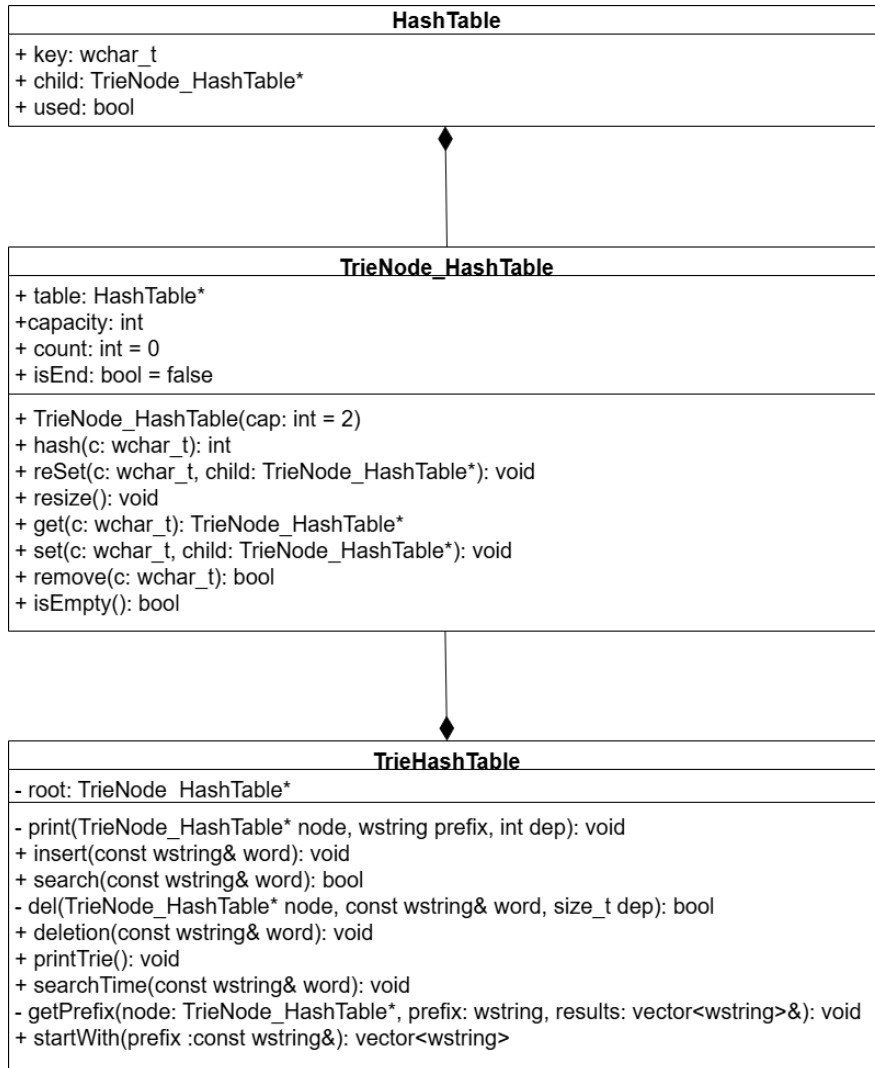


Figure 2.9: UML class diagram of Hash Table Trie

Chapter 3

Experimental Setup and Details

3.1 Overview

This section will mention all the experimental setups used to evaluate the performance of three kinds of variants of trie data structure. It includes the machine configuration, the datasets used for testing, the compiler flags, encoding and timing setup.

The experiment mainly focuses on testing the performance of three different of trie data structures: nonsorted array, sorted array and hash table trie. The goal is that to evaluate their behavior under various input conditions from simple words like "cat", "dog", "team" to complex Unicode strings (ex: Vietnamese, Chinese, Czech) like "áo dài", "丰富", "kočka".

In addition, to deeper understand their performance, the experiment also used entire collection of words which means these trie data structures will have to insert a set of words with varying lengths and character positions, and the length of each words will be evaluated also.

All datasets are encoded in UTF-8, supporting over 1.1 million Unicode code points [17, 18], let $|S| = 1,114,112$, which means a node can handle maximum $|S|$ child nodes. ASCII (with only 128 characters) is treated as a special case.

3.2 Experimental Setup

3.2.1 Machine Configuration

The experiments were run on a machine with stable and consistent configuration to ensure that the fair comparison between all variants in this experiment. The hardware and software setup used through the testing is detailed in Table 3.1:

Component	Specification
Processor	Intel® Core™ i7-10750H @ 2.60GHz
RAM	32 GB
Operating System	Windows 11 Pro 64-bit
IDE	Visual Studio Code 1.98.2 (64-bit)
Compiler	GCC (g++) version 13.2.0 (Rev2, built by MSYS2 project)
Build Tool	Custom Makefile

Table 3.1: Machine and software configuration used in the experiment

3.2.2 Datasets

In this setup, the datasets were used has different size from minimum is at least 72,000 words to more than 1,400,000 words and different character positions. Each dataset contains it own language to easy to evaluate the performance of each trie data structure. The datasets were downloaded on Github and manually created.

Dataset	Description	Number of words	Unicode	Source
english.txt	Sample of common English words, some contain numbers and special characters.	466,550	No	[19]
viet.txt	A collection of Vietnamese words which also contains numbers and special characters.	72,547	Yes	[20]
czech.txt	A list of Czech words. Many words contain Czech orthography.	535,889	Yes	[21]
chinese.txt	The dataset includes words with complex Chinese characters and no Latin letters.	406,590	Yes	[22]
mix.txt	A list of all words from previous datasets to test the implementation.	1,481,576	Yes	Manually created

Table 3.2: Summary of datasets used in the experiment

Although the UTF-8 supports up to $|S| = 1,114,112$, but the actual size of each dataset is different:

- **english.txt**: $|S| \approx 100$, standard ASCII characters, numbers, and symbols
- **viet.txt**: $|S| \approx 165$, Latin characters, Vietnamese diacritics, numbers, and punctuation

- `czech.txt`: $|S| \approx 135$, includes Latin base, Czech accents, numbers, and symbols
- `chinese.txt`: $|S| \approx 3,300$, mostly Chinese ideographs and some punctuation
- `mix.txt`: $|S| \approx 3,800$, union of all characters across datasets

Table 3.3 presents the word length statistics for each dataset used in the experiment. It includes the minimum, average and maximum word lengths based on 5 datasets in Table 3.2. The different in the word length is important for evaluating the performance of trie data structures.

Dataset	Minimum Length	Average Length	Maximum Length
<code>english.txt</code>	1	9	45
<code>viet.txt</code>	1	3	17
<code>czech.txt</code>	1	9	27
<code>chinese.txt</code>	1	3	23
<code>mix.txt</code>	1	7	45

Table 3.3: Word length statistics for each dataset

3.2.3 Build and Compilation Setup

All implementations of trie variants in this study were compiled using a Makefile and execute using the `c++` compiler (`g++`). It will ensure that the testing will run safely during compilation and the build configuration included flags:

- `-std=c++23`: This flag support the C++ 23 features and improve syntax for modern C++.
- `-Wall`: Notify the warnings, helping to detect issues.
- `-municode`: because the experiment will work with Unicode, hence, this flag will ensure our windows can run Unicode.

The Makefile was designed to compile all `“.cpp”` files into object files (`“.o”`) and link them to a executable file (`“.exe”`) It also includes a clean rule to remove all object files, helping to quick rebuilt the experiment.

3.2.4 Encoding and Timing Setup

All the datasets are encoded in UTF-8, In this implementation, input datasets are opened using `wifstream` with the `codecvt_utf8<wchar_t>` facet, UTF-8 encoded text to be automatically decoded into wide characters (`wchar_t`) on read. Also, all words are processed using `std::wstring`, ensuring that each character is treated as a code point.

Execution time for operations were measured by using `std::chrono::high_resolution_clock`. All timings were recorded in seconds.

3.3 Experimental Details

In this experiment, a query is defined as a single input that processed by a an operation such as insertion, search or deletion. Each query represents to an unique full word (or string) or a single character from datasets at table 3.2. For example, searching 500,000 words means processing 500,000 queries. To better understand the structure and performance of different trie data structures, characteristics of Tries will be proposed:

- The height of a trie depends on the lengths of the words in the dataset. The minimum height is 1. The average height reflects the average word length in the dataset, and the maximum height equals the length of the longest word.
- The average number of children per node $|C|$ in a trie structure is 1 across datasets. This is because the characteristic of the trie is built from a large alphabet dataset. While the root is branching due to common prefixes, most nodes at the deeper levels have only 1 child. Each word, after splitting from shared prefixes, it continues to traversal its own path without splitting. As a result, the number of children closely matches the total number of nodes, leading to an average $|C|$ value close to 1. This observation is further illustrated in the histograms of $|C|$ across datasets, see Figures 3.1, 3.2, 3.3, 3.4, and 3.5, where the majority of nodes have either 0 or 1 child. In all datasets, the number of nodes with $|C| = 1$ dominates, confirming that most paths in the trie are linear. Although there are nodes with higher branching ($|C| > 5$), they account for a small percentage of the total nodes, usually only occur near the root.

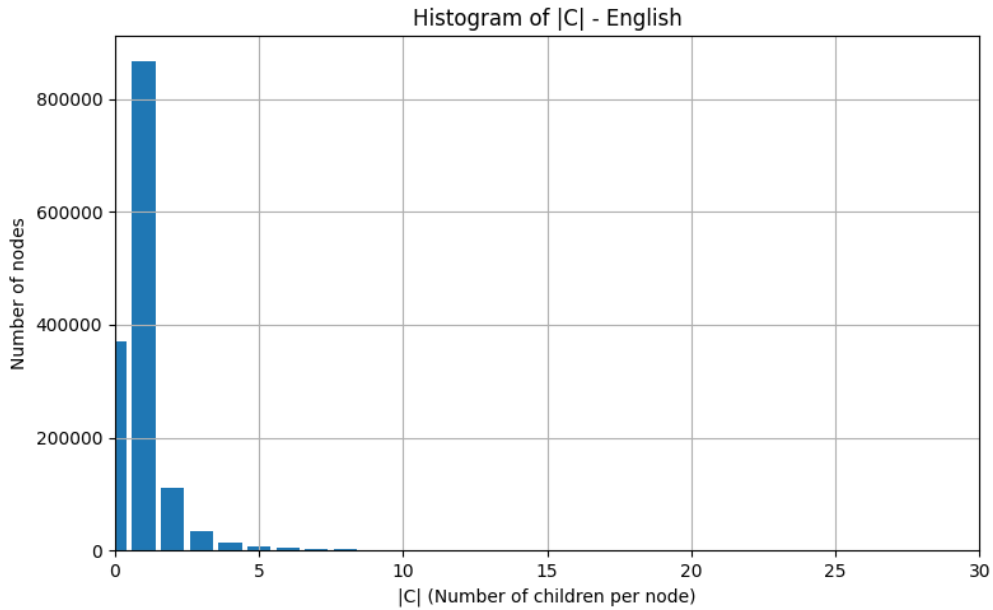


Figure 3.1: Histogram of $|C|$ in English dataset

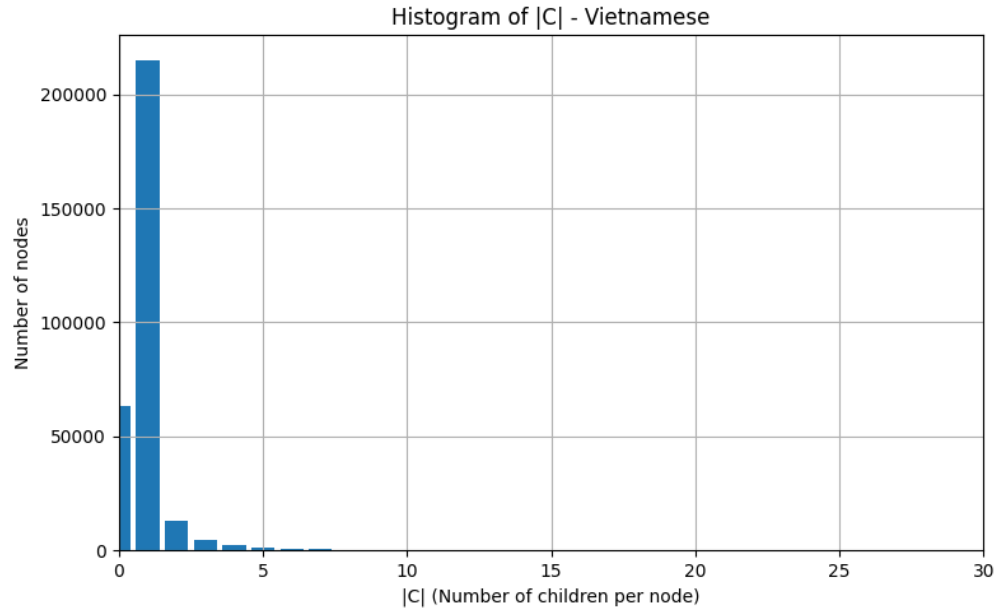


Figure 3.2: Histogram of $|C|$ in Vietnamese dataset

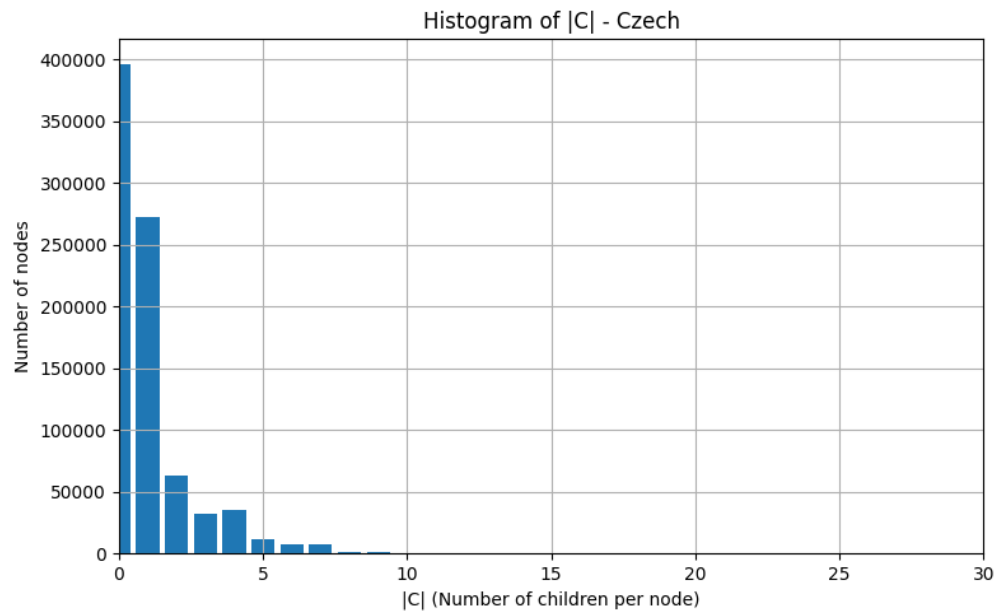


Figure 3.3: Histogram of $|C|$ in Czech dataset

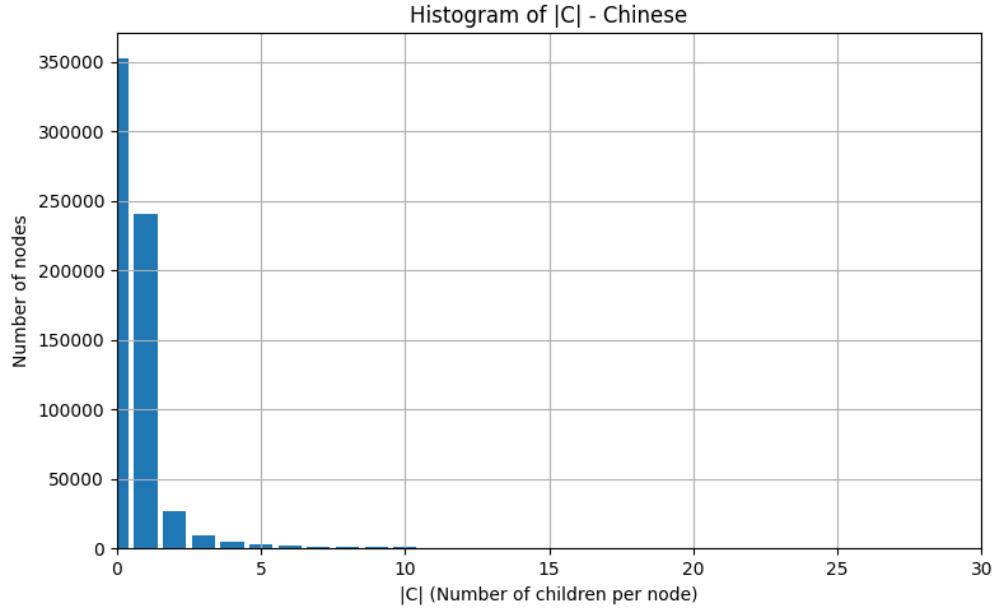


Figure 3.4: Histogram of $|C|$ in Chinese dataset

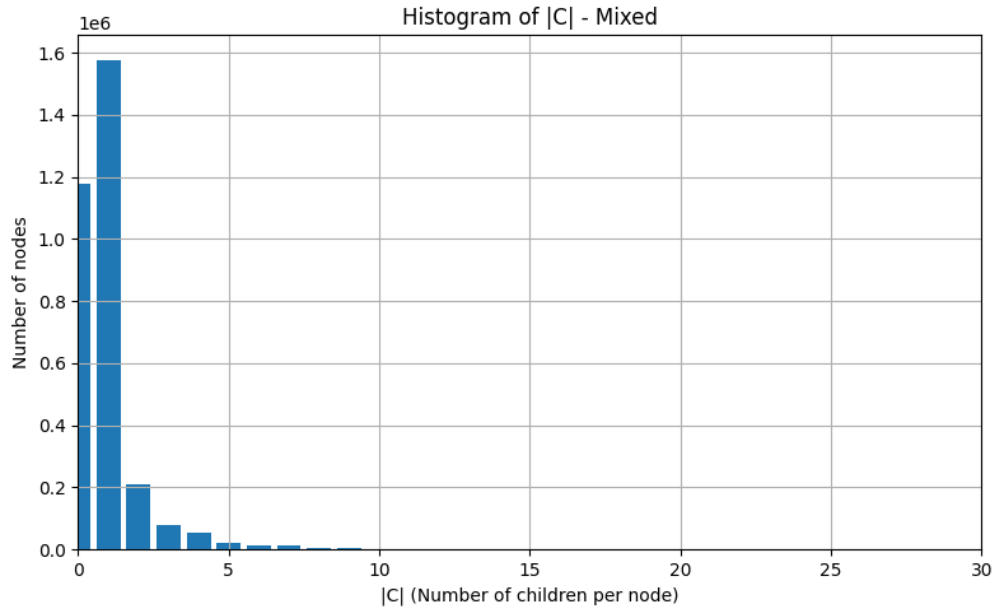


Figure 3.5: Histogram of $|C|$ in Mixed dataset

The experiment is also included a sequential array structure, this will store all words in a flat array, Unlike trie structures are presented in Chapter 2, it does not use any prefix branching, all operations is sequential array such as insertion, search, and deletion are used linear scans with

$|S| = n$, where n is the number of words in the datasets. This comparison helps highlight strengths and weaknesses of other trie data structures.

3.3.1 Insertion Experiment

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	466,550	0.84	0.53	0.37	0.12
viet.txt	72,547	0.17	0.09	0.07	0.02
czech.txt	535,889	0.69	0.37	0.29	0.18
chinese.txt	405,590	5.38	0.51	0.26	0.10
mix.txt	1,481,576	8.15	1.45	0.98	0.41

Table 3.4: Insert time (in seconds) for different trie types and sequential array across datasets

Table 3.4 shows the insertion time (in seconds) of three trie data structures: Non-Sorted Array, Sorted Array and Hash Table and a simple Sequential Array on five different datasets. In the data, Sequential Array demonstrates the fastest insertion due to lack of overhead, it simply adds into the array. Among trie structures, the Hash Table Trie outperforms others in every dataset, proving that Hash Table is efficiency in storing and managing different and large character sets, especially Unicode. The Sorted Array has benefit from binary search before inserting, improving speed compared to Non-Sorted Array which uses linear search for inserting characters.

In specific observations, dataset **english.txt** shows Sequential Array obtains 0.12 seconds, while Hash Table Trie reaches 0.37 seconds, Sorted Array 0.53 seconds, and Non-Sorted Array 0.84 seconds. For **viet.txt**, all structures return fast results due to the number of words is small, Hash Table again leads among tries with 0.07 seconds. On **czech.txt**, although it has a complex character structures, Hash Table still gets 0.29 seconds.

In **chinese.txt**, which contains a wide range of Unicode characters, the Non-Sorted Array gives a poorly result 5.38 seconds which means it has a limitation in handling high branching nodes. In contrast, Hash Table returns a low insertion time 0.26 seconds, highlighting its strength. Finally, in **mix.txt**, which combines all datasets, the Non-Sorted Array reaches 8.15 seconds while Hash Table still gives a great result 0.98 seconds, and Sequential Array leads with 0.41 seconds.

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	466,550	105.68	105.02	140.50	26.36
viet.txt	72,547	22.91	22.79	29.96	3.81
czech.txt	535,889	59.79	59.23	89.45	29.48
chinese.txt	405,590	47.45	47.30	75.04	13.38
mix.txt	1,481,576	234.61	233.55	333.39	72.33

Table 3.5: Insert memory (in MBs) for different trie types and sequential array across datasets

Table 3.5 reports the memory usage (in megabytes) of the same four data structures used in the insertion. The statistics show that Hash Table while gives a great results in time, it consumes more memories compared to other structures, because it has too many empty array slots to maintain low load factors during insertion and linear probing mechanism to solve collisions.

On the other hand, the Sequential Array consumes the least memory across the datasets, after that are Non-Sorted and Sorted Array tries, which gives almost the same results, which is because both are using the same structure and the binary search in the Sorted Array does not incur additional memories.

In detailed analysis, for the dataset `english.txt`, dataset with 466,550 queries, the Hash Table consumes 140.50 MB, which is about 1.3 times the memory compared to Non-Sorted Array 105.68 MB and Sorted Array 105.02 MB, and over five times of the Sequential Array 26.36 MB, and it occurs same with other datasets

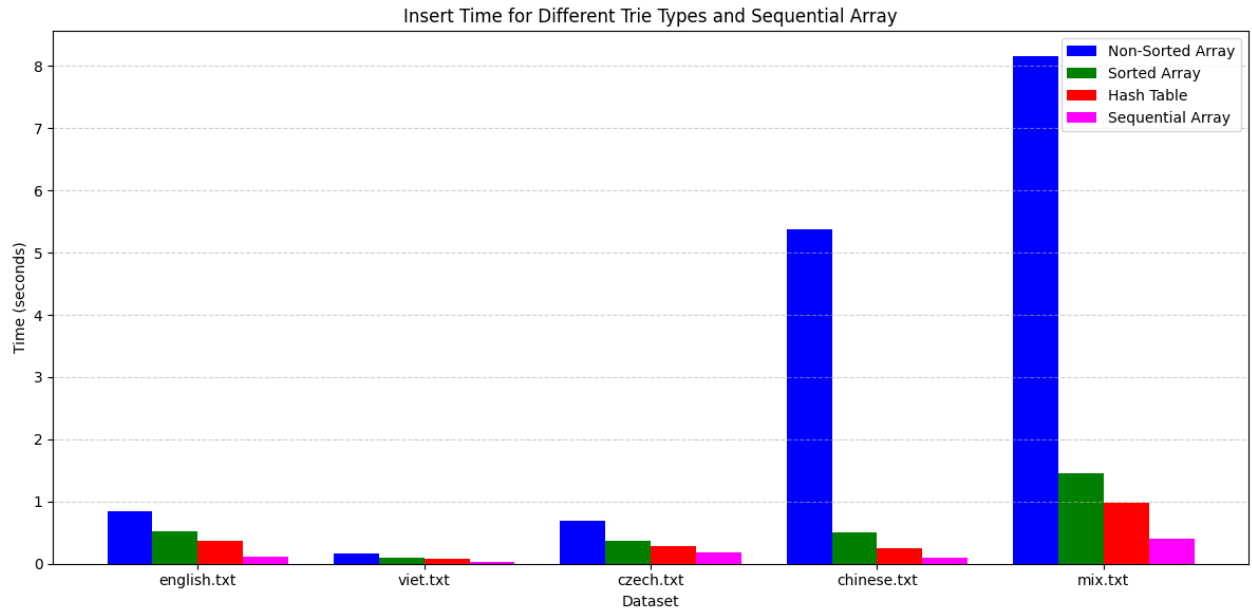


Figure 3.6: Insert time (in seconds) across datasets

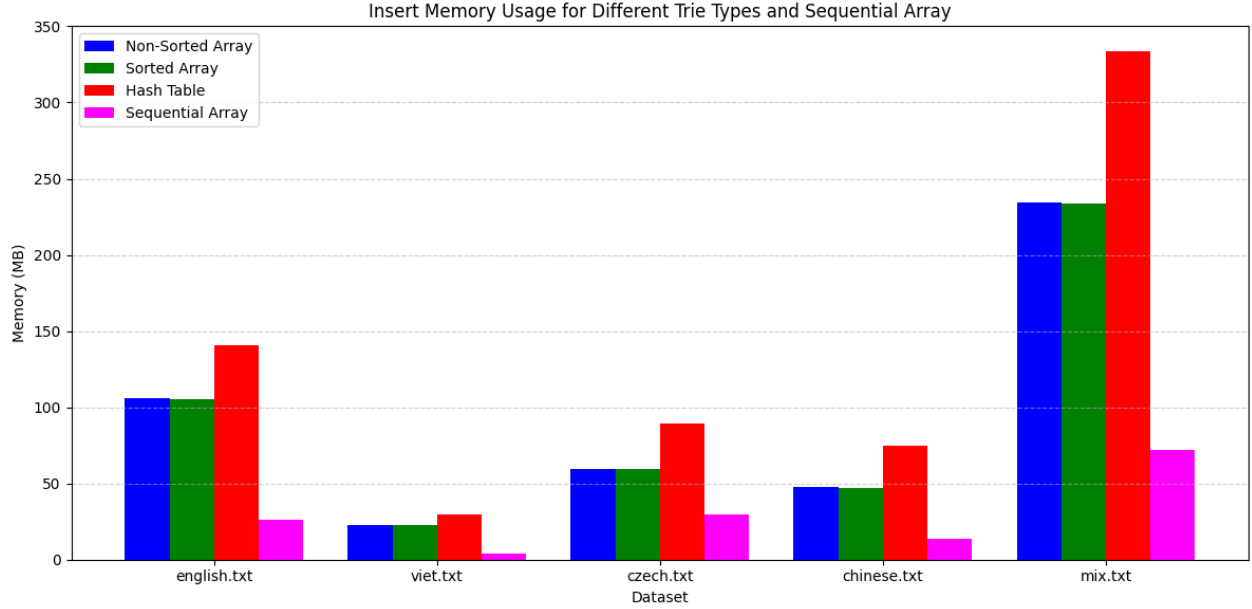


Figure 3.7: Insert memory usage (in MBs) across datasets

In summary, the insertion experiment reveals the different between speed and memory across data structures. The Sequential Array gives the fastest insert time and consumes the least memories, due to its simplicity in insertion. The Hash Table outperforms other trie data structures in speed experiment, see Figure 3.6, especially in `chinese.txt` and `mix.txt` datasets. However, to archive this speed, it costs a significant memory, see Figure 3.7. The Sorted Array performs better than the Non-Sorted Array trie, in both time and memory usage. The speed comes from the benefit of binary search which improves Sorted Array speed and also, this does not gain any memory usage. In Non-Sorted Array, since it uses linear scan which decrease the speed of this data structure, see Figure 3.6. Overall, the large memory consumption of trie structures is mainly due to the overhead of storing too many child nodes and create many slots unused. To solve this problem, several approaches can be used such as: Allocating child arrays or tables based on actual usage instead of initializing large capacities, for example, in hash table, we can decrease the capacity to make it less memory wasted. The second approach is that uses a compressed trie, which type of trie compressing redundant node into one node, helping saving space and reduce the memory wasted. The experiments demonstrate that if memory is limited, the Sequential Array or Sorted Array are better choices. In contrast, if speed is important and memory is not a concern, the Hash Table is preferable.

3.3.2 Search Experiment

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	10,000	0.019	0.013	0.011	35.140
viet.txt	10,000	0.019	0.010	0.009	4.770
czech.txt	10,000	0.016	0.012	0.010	42.261
chinese.txt	10,000	0.132	0.007	0.005	35.109
mix.txt	10,000	0.056	0.013	0.011	105.800

Table 3.6: Search time (in seconds) for different trie types and sequential array across datasets

Table 3.6 presents the time of searching 10,000 words in five datasets using four data structures. The results clearly show that trie structures outperform the Sequential Array.

The Hash Table provides the fastest search across all datasets. See Table 3.6, it takes only 0.011 seconds for `english.txt`, 0.009 seconds for `viet.txt`, and as well as 0.005 seconds for `chinese.txt`, which is the most complex and largest Unicode character set.

The Sorted Array performs better than the Non-Sorted Array in all cases due to the binary search on sorted child nodes. In `english.txt`, it reaches 0.013 seconds instead of Non-Sorted Array which has 0.019 seconds.

The Non-Sorted Array performs well but it is the slowest across three trie data structures due to linear scan over children, It takes 0.132 seconds on `chinese.txt`, showing its limitation in searching words with high branching.

The Sequential Array performs the worst in all datasets. With `mix.txt`, it takes 105.800 seconds, and even for smaller dataset like `viet.txt`, it still obtains 4.770 seconds.

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
english.txt	pne	0.00032	0.00029	0.00049	0.01242
viet.txt	cái	0.00013	0.00012	0.00022	0.00200
czech.txt	nez	0.00104	0.00101	0.00183	0.01404
chinese.txt	人	0.00061	0.00058	0.00116	0.00796
mix.txt	in	0.01114	0.01111	0.02088	0.03673

Table 3.7: Prefix search time (in seconds) across datasets and data structures.

Table 3.7 presents the prefix search time across datasets using four data structures: Non-Sorted Array Trie, Sorted Array Trie, Hash Table Trie, and Sequential Array. Each query retrieves a list of all strings that begin with the specified prefix. The Non-Sorted Array Trie performed closely to the Sorted Array Trie in all cases, with differences in the order of microseconds. This confirms that even a simple linear-scan approach can achieve high performance for prefix searches.

The prefix search results reveal that Sorted Array Trie achieved the fastest performance across all

datasets. For instance, in `english.txt` with prefix "pne", it completed the query in 0.00029s, while the Hash Table Trie was 69% slower, and Sequential Array was 42 times slower.

Similarly, In `viet.txt` with prefix "cái" and `czech.txt` with prefix "nez", where Hash Table Trie was 83% and 81% slower than the Sorted Array Trie. In `chinese.txt` with prefix "人", the hash table took nearly twice the time, and in `mix.txt` with prefix "in", a prefix also presents in `english.txt`, `viet.txt`, and `czech.txt`, the Sequential Array was over three times slower.

In conclusion, trie structures bring a fast search speed on all datasets, with the Hash Table Trie being the fastest and most effective structure. The Sorted Array Trie provides a good balance between performance and simplicity because of binary search. Although the Sequential Array is a simple structure, it is unsuitable for searching in large datasets with various characters. These findings highlight the strength of searching in trie structures compared to Sequential Array. However, it is observed that in case of searching prefixes, the search time of the Hash Table Trie increases significantly. This performance problem is due to the use of linear probing in handling collisions, which leads to elements in the hash table to cluster together and takes more probing steps to find the right position when the hash table is full. To solve this problem, the common solution is that using quadratic probing is a collision resolution technique that searches for the next available slot by jumping further away in each step using a squared offset, for example, it will use $i^2 = 1, 2, 4, \dots$ instead of using only number 1 in linear probing, see Equation 2.2.

3.3.3 Deletion Experiment

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Sequential Array
<code>english.txt</code>	10,000	0.030	0.021	0.017	232.709
<code>viet.txt</code>	10,000	0.035	0.019	0.015	32.196
<code>czech.txt</code>	10,000	0.026	0.017	0.015	274.490
<code>chinese.txt</code>	10,000	0.245	0.016	0.008	218.652
<code>mix.txt</code>	10,000	0.104	0.021	0.017	735.939

Table 3.8: Deletion time (in seconds) for different trie types and sequential array across datasets

Table 3.8 presents the deletion time (in seconds) for 10,000 operations across five datasets and four structures. The results reveal the same results in section 3.3.2: trie structures perform deletion much faster than the Sequential Array.

The Hash Table gives the best performance in deletion, especially with complex datasets like `chinese.txt`, where it only takes 0.008 second. This proves that the direct access child nodes of the hash table helps determine and delete the node quickly.

The Sorted Array again outperforms the Non-Sorted Array, due to use the binary search to navigate children during traversal. In `viet.txt`, the Sorted Array Trie only takes 0.019 seconds, while the

Non-Sorted takes 0.035 seconds.

The Non-Sorted Array shows the time higher in deletion time due to linear scan through child nodes. It is more obvious in the `chinese.txt` dataset with 0.245 seconds, where the number of branches per node is higher.

The Sequential Array performs extremely worst for delete operation. It requires complete string matching and then starting to scan at the begin of the array again, leading to very high time up to 735.939 seconds in `mix.txt` dataset for 10,000 words.

In conclusion, trie structures provide highly efficient deletion performance across all datasets, with the Hash Table Trie is the fastest structure. The Sorted Array Trie is again a balanced option due to its traversal via binary search. In contrast, the Sequential Array is clearly not suitable for deleting in large alphabets or high branching datasets.

3.3.4 Preliminary Results

This section provides a preliminary evaluation of solutions proposed to address the problems identified in earlier experiments. Based on the limitations observed in memory usage and search performance of the initial trie implementations, two enhanced approaches are explored: compressed tries for reducing memory, and quadratic probing for improving prefix search efficiency in Hash Table trie.

3.3.4.1 Compressed Trie

To address the issue of large memory usage in standard tries, the preliminary experiment using a compressed trie need to conduct. The working principle of Compressed Trie is based on compressing the nodes of a standard Trie. In Compressed Trie, when a node has only one child branch, the character string from that node to the child node will be merged into a single node to save space [23]. Table 3.9 shows the efficiency of Compressed Trie across different datasets.

Dataset	Queries	Non-Sorted Array	Sorted Array	Hash Table	Compressed Trie
<code>english.txt</code>	466,550	105.68	105.02	140.50	65.24
<code>viet.txt</code>	72,547	22.91	22.79	29.96	11.01
<code>czech.txt</code>	535,889	59.79	59.23	89.45	63.32
<code>chinese.txt</code>	405,590	47.45	47.30	75.04	46.00
<code>mix.txt</code>	1,481,576	234.61	233.55	333.39	184.46

Table 3.9: Insert memory (in MBs) for different trie types across datasets

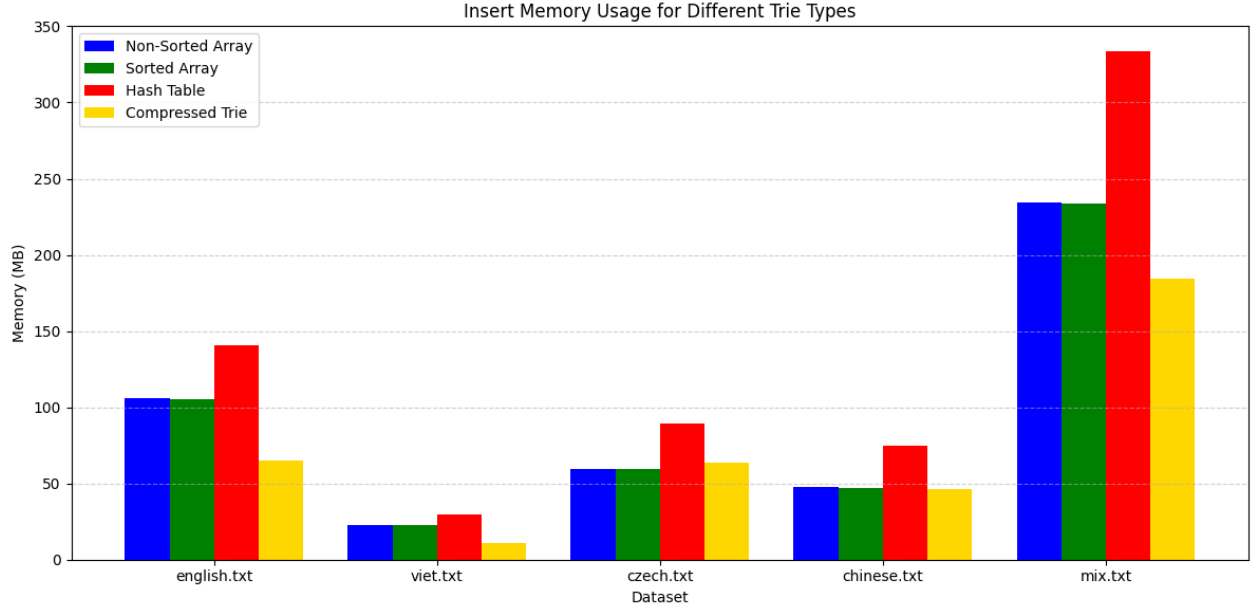


Figure 3.8: Insert memory (in MBs) across datasets

Table 3.9 and Figure 3.8 illustrate the memory consumption (in megabytes) of different trie structures across multiple datasets. The compressed trie achieves the lowest memory usage compared to other implementations, confirming its effectiveness in saving space.

Specifically, for the `english.txt` and `viet.txt` datasets, the compressed trie reduces memory consumption by approximately 38% and 48%, respectively, compared to the non-sorted array trie. For larger datasets such as `mix.txt`, the compressed trie saves around 21% of memory. Even in the `czech.txt` and `chinese.txt` datasets, where the structures share more prefixes, compressed tries still obtains a significant reduction in memory consumption.

Overall, the experimental results clearly demonstrate that compressed tries are highly effective in saving the memory in large alphabets by merging unique suffixes into single nodes.

3.3.4.2 Quadratic Probing

To solve the problem of slow prefix search performance in the Hash Table Trie observed in Table 3.7, quadratic probing is used as a collision resolution technique. This method spreads entries more uniformly across the hash table by increasing the probe distance quadratically with each collision [24], thus reducing clustering and significantly improving search efficiency, it can be proposed based on the equation [24]:

$$index = \left(hash(code_point) + i^2 \right) \bmod capacity \quad (3.1)$$

Dataset	Prefix	Hash Table (Linear)	Hash Table (Quadratic)
english.txt	pne	4.90×10^{-4}	2.50×10^{-4}
viet.txt	cái	2.20×10^{-4}	1.60×10^{-6}
czech.txt	nez	1.83×10^{-3}	1.03×10^{-3}
chinese.txt	人	1.16×10^{-3}	7.00×10^{-7}
mix.txt	in	2.09×10^{-2}	9.63×10^{-3}

Table 3.10: Prefix search time (in seconds) comparing linear and quadratic probing.

Table 3.10 highlights key improvements when replacing linear probing with quadratic probing in the Hash Table Trie. In `viet.txt` and `chinese.txt`, quadratic probing significantly reduced prefix search time by over 99%, indicating its strong advantage on datasets with high collision rates or large character sets. On `mix.txt`, which contains multilingual data, the search time dropped by nearly 54%, showing that quadratic probing operates more effectively when processing complex data sets.

Overall, quadratic probing improves efficiency across datasets, especially where collisions and probing chains occur continuously.

Chapter 4

Conclusion

In this thesis, Trie data structures have been presented and analyzed in detail for efficient string storage and retrieval across multilingual datasets. Beginning with the concepts of basic trie operations such as insertion, search and deletion, the work expanded to node structures of Non-Sorted Array, Sorted Array, Hash Table and Sequential Array as a comparison data structure.

To evaluate performance, multiple datasets in different languages such as English, Vietnamese, Czech, Chinese and a combined mixed dataset were used. Experimental results showed clearly between speed and memory usage depending on the data structure applied on trie. The Hash Table Trie returned the fastest results in all basic operations, especially for large and complex datasets, but consumed more memories. In contrast, Sequential Array was the most memory saving structure, but its linear structure made the search and deletion times slowdown.

The Sorted-array Trie is the most balance structure due to binary search, making it suitable for applications needing both speed and saving memories. Meanwhile, the Non-Sorted Array provided a simple algorithm, however, it proved less efficient with large datasets.

In conclusion, this thesis confirms that trie data structures are powerful in processing with strings. Depending on requirements such as speed or memory different trie implementations can be chosen to get the most optimal performance.

Bibliography

1. GEEKSFORGEES. *Trie Data Structure Tutorial* [<https://www.geeksforgeeks.org/introduction-to-trie-data-structure-and-algorithm-tutorials/>]. [N.d.]. Published: 2024-08-07.
2. KALITA, Rishiraj. *Trie Data Structure* [<https://www.scaler.in/trie-data-structure/>]. [N.d.]. Published: 2024-09-26.
3. DESIGN GURUS. *Introduction to Trie*. 2025. Available also from: <https://www.designgurus.io/course-play/grokking-data-structures-for-coding-interviews/doc/introduction-to-trie>.
4. KNUTH, Donald E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd. Addison-Wesley, 1998.
5. FREDKIN, Edward. Trie Memory. *Communications of the ACM*. 1960, vol. 3, no. 9, pp. 490–499.
6. GUSFIELD, Dan. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.
7. CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
8. JUVILER, Jamie. *What is UTF-8 Encoding? A Guide for Non-Programmers*. 2025. Available also from: <https://blog.hubspot.com/website/what-is-utf-8>.
9. JAISINGH, Anand. *Trie (Keyword Tree) - Tutorial* [<https://www.hackerearth.com/practice/data-structures/advanced-data-structures/trie-keyword-tree/tutorial/>]. [N.d.]. Accessed: 2024-10-22.
10. TAILOREDREAD AI. *Mastering Trie Data Structures: Optimizing Time Complexity in Computer Science and Algorithms*. TailoredRead, 2025. Available also from: <https://tailoredread.com/book/mastering-trie-data-structures-optimizing-time-32ef9e75ba5a>.
11. GEEKSFORGEES. *Trie Memory Optimization using Hash Map*. 2023. Available also from: <https://www.geeksforgeeks.org/trie-memory-optimization-using-hash-map/>. Last Updated: 16 Mar, 2023.

12. GOLDMAN, Sally A.; GOLDMAN, Kenneth J. *A Practical Guide to Data Structures and Algorithms using Java*. Chapman & Hall/CRC, 2007. ISBN 978-1-4200-9381-0. Available also from: <https://www.taylorfrancis.com/books/mono/10.1201/9781420010336/practical-guide-data-structures-algorithms-using-java-sally-goldman-kenneth-goldman>.
13. AOE, Jun-ichi; MORIMOTO, Katsuhiko; SATO, Toshihiro. An efficient implementation of trie structures. *Software: Practice and Experience*. 1992, vol. 22, no. 9, pp. 695–721. Available from DOI: 10.1002/spe.4380220902.
14. MDN CONTRIBUTORS. *Code point - MDN Web Docs Glossary*. 2024. Available also from: https://developer.mozilla.org/en-US/docs/Glossary/Code_point.
15. WANG, Thomas. *Integer Hash Function* [<https://gist.github.com/badboy/6267743>]. 2013.
16. CAFIERO, Clayton. *Linear Probing* [https://www.uvm.edu/~cbcafier/cs2240/content/10_probing/01_linear.html]. 2025-01.
17. CONSORTIUM, Unicode. *The Unicode Standard, Version 15.1*. Unicode, Inc., 2023. Available also from: <https://www.unicode.org/versions/Unicode15.1.0/>.
18. GEEKSFORGEEKS. *What is Unicode?* 2024. Available also from: <https://www.geeksforgeeks.org/what-is-unicode/>.
19. DWYL. *English Words* [<https://github.com/dwyl/english-words>]. [N.d.].
20. LE, Duyet. *Vietnamese Wordlist* [<https://github.com/duyet/vietnamese-wordlist>]. [N.d.].
21. KKRYPT0NN. *Czech Wordlist* [<https://github.com/kkrypt0nn/wordlists/tree/main>]. [N.d.].
22. MOE, Lwin. *Chinese Word Segmenter* [<https://github.com/lwinmoe/segment/tree/master>]. [N.d.].
23. GEEKSFORGEEKS. *Compressed Tries - GeeksforGeeks*. 2024. Available also from: <https://www.geeksforgeeks.org/compressed-tries/>. Published: 11 Apr, 2024.
24. TOPICS, Scaler. *Quadratic Probing*. n.d. Available also from: <https://www.scaler.in/quadratic-probing/>.

Appendix A

Command-Line Interface

This appendix describes how to execute the trie benchmark application from the command line.

Build Instructions

- **Platform:** Windows (uses Windows-specific headers like `<Windows.h>`, `<psapi.h>`, `_setmode`, etc.)
- **Compiler:** Ensure you have a C++ compiler that supports C++23 or newer.

Make sure the following source files are present:

- `main.cpp`
- `nonsortedTrie.cpp`
- `sortedTrie.cpp`
- `hashtable.cpp`
- `sequentialArray.cpp`
- `sortedCompressed.cpp`
- `hashQuadratic.cpp`

Makefile Commands

To compile and clean the application, use the following commands:

- `make`
Compiles all source files and produces the executable.

- `make clean`

Removes object files and the executable to allow a fresh rebuild.

Usage Syntax

```
trieapp <data_file> <command> <word_or_number>
```

Supported Commands

- `-s <word>`
Search for the specified word in all trie types.
- `-s -r <number>`
Randomly selects `<number>` words from the input file and searches them.
- `-s -pr <prefix>`
Searches all words that begin with the given prefix.
- `-d <word>`
Deletes the specified word from all tries.
- `-d -r <number>`
Randomly deletes `<number>` words from the tries.
- `-p <option>`
Prints the trie structure. Available options:
 - 1: Non-Sorted Array Trie
 - 2: Sorted Array Trie
 - 3: Hash Table Trie

Examples

```
trieapp data/test.txt -s world
trieapp data/test.txt -d hello
trieapp data/test.txt -s -r 10000
trieapp data/test.txt -d -r 10000
trieapp data/test.txt -s -pr pre
trieapp data/test.txt -p 2
```

Program Output

The program outputs the time (in seconds) for each operation, and the memory used (in megabytes) for insertion. For random modes, it verifies and reports if any word failed during the process.